

RTL Design Sherpa

Ocelus White Paper Reconstructing What Happened — Rev 0.9

October 2026

Table of Contents

1 Ocelus: Reconstructing What Happened....5

2 Overview: from an SoC failure to a usable history....6

2.1 The five planes....8

2.2 An implementation, end to end....9

2.3 What the integrator owns....11

3 The spine....12

4 Which monitor: the lite, unless you can say why not....14

5 Identity space allocation....16

6 Where to insert monitoring....18

7 Timestamp policy....20

8 Drain path selection....22

9 Packet-type filtering....25

10 Aggregation topology....27

11 What it costs....28

12 Validating a tweak in simulation....30

13 Out of scope....31

14 Relationship to Nexus (IEEE-ISTO 5001)....32

15 References....34

15.1 Interconnect monitoring (the closest relatives)....35

15.2 Debug and trace standards....36

15.3 Debug-infrastructure surveys and tracers....37

15.4 Time and ordering....38

15.5 Structures the design converges with....39
16 RTL AMBA Modules Index....40
16.1 Overview....41
16.2 AMBA 4 Protocol Implementations....42
16.2.1 WB4 (Wishbone B4, pipelined)....42
16.2.2 APB4 (Advanced Peripheral Bus)....42
16.2.3 AXI4-Full (Memory-Mapped Interface)....43
16.2.4 AXI4-Lite (Register Interface)....44
16.2.5 AXI4-Stream (Streaming Interface)....44
16.3 AMBA 5 Protocol Implementations....45
16.3.1 APB5 (Advanced Peripheral Bus - AMBA 5)....45
16.3.2 AXI5 (High-Performance Memory-Mapped - AMBA 5)....45
16.3.3 AXI5-Stream (Streaming Interface - AMBA 5)....46
16.3.4 ACE (AXI Coherency Extensions)....47
16.3.5 CXS (Credited eXtensible Stream)....47
16.4 Shared Infrastructure and Utilities....49
16.4.1 Generic AXI (GAXI) Components....49
16.4.2 Arbitration and Control....49
16.4.3 AXI Monitoring and Analysis....49
16.4.4 Monitor Bus Delivery + Bulk-Trace Compression....50
16.4.5 Memory / BRAM Slave (sdpram_slave family)....51
16.4.6 Address and Data Processing....51
16.4.7 Clock and Reset Management....51
16.5 Protocol Bridge and Shims....52

16.5.1 AXI4 to APB Conversion....52
16.6 Ocelus....53
16.7 Package Definitions....54
16.7.1 Documented packages....54
16.7.2 The other two packages are documented, just not under their own names....54
16.8 Quick Reference....55
16.8.1 Module Count by Protocol....55
16.8.2 Performance Characteristics....55
16.8.3 AMBA 4 vs AMBA 5 Selection Guide....56
16.9 Usage Guidelines....57
16.9.1 Protocol Selection....57
16.9.2 Clock Gating Benefits....57
16.9.3 Design Integration....57
16.10 References....58
16.11 Navigation....59

1 Ocelus: Reconstructing What Happened

Audience: SoC integrators deciding how to instrument a design with the RTL Design Sherpa monitor system – branded Ocelus. **Not** a status snapshot of what is in the tree (that is overview.md) and not a port list (those are the per-module pages under monitor/). This paper starts with an SoC failure and follows the evidence: how passive monitors turn independent symptoms into an ordered error history, how a shared microsecond time base connects events across the chip, and how the five fabric planes deliver that history to software. It then works through a real implementation end to end, the axes the integrator owns, and what each choice costs.

Version: 0.9, 2026-10-26. Ocelus and this paper remain at version 0.9 until Ocelus has been validated in silicon. Numbers are from the tree at 2026-10-12; each is cited to the page or report it comes from. The paper recommends the lite monitor on every port, with the full monitor as the exception requiring justification. It frames Ocelus as five planes (protocol, error, debug, perf, output), connects them through an SoC incident and a memory-resident error history, distinguishes event order from drain order, and presents the proposed global usync time base.

---

2 Overview: from an SoC failure to a usable history

An SoC failure usually arrives as a sequence, not as one isolated error. A malformed descriptor sends a DMA transaction to an unmapped address. The bridge rejects it. The DMA later times out waiting for completion, and the stream waiting for that data times out in turn. Every monitor has seen something real, but three disconnected alarms leave software to guess which event came first and which are consequences.

Now give those events a common microsecond time base and collect the error records in a reserved memory region:

Time (µs)	Source	Event
18,442,731	AXI bridge	Address decode error
18,442,935	DMA channel 2	Completion timeout
18,443,002	Output stream	Data timeout

The list does not prove causality by itself. It does something immediately useful: it puts the earliest observed fault in front of the engineer, preserves the sequence that followed, and associates each record with its source. The investigation starts with the bridge and the descriptor instead of with three unrelated interrupt handlers. A history in memory is also available for postmortem capture when no live host can service each event as it arrives.

That is the SoC-level purpose of Ocelus. The monitors, timestamps, identities, aggregation tree, and drains are not ends in themselves; together they turn distributed symptoms into evidence software can read in context.

Ocelus — from the Latin *ocellus*, "little eye"; a bee's simple always-on eye — is the repository's monitor fabric: the blocks, the packet format, and the drain machinery that watch a design while it runs and report what crosses the conditions you set. It is **not limited to tracking simple interfaces**, however the earlier drafts of this paper read. The port monitors are its most visible producers, but the fabric equally carries:

- **Protocol events** from passive taps on any handshake: AXI4/AXI5, AXI-Lite, AXI-Stream, CXS, APB, Wishbone — full and lite monitors per port family.
- **In-core events** from blocks that emit their own: STREAM's descriptor engine and scheduler carry lite monitors *inside* the DMA core; RAPIDS monitors its descriptor and AXIS interfaces the same way; the bridge generator puts one on every monitored port.
- **In-datapath measurement** that needs no transaction tracking at all: the `axi_bus_meter` counts beats, bytes and idles in the flow, and the pass-through observer watches a mid-fabric link with its own configuration.

- **Error, debug and performance classes** as first-class packet types with their own reporters, not as afterthoughts of a protocol trace.

2.1 The five planes

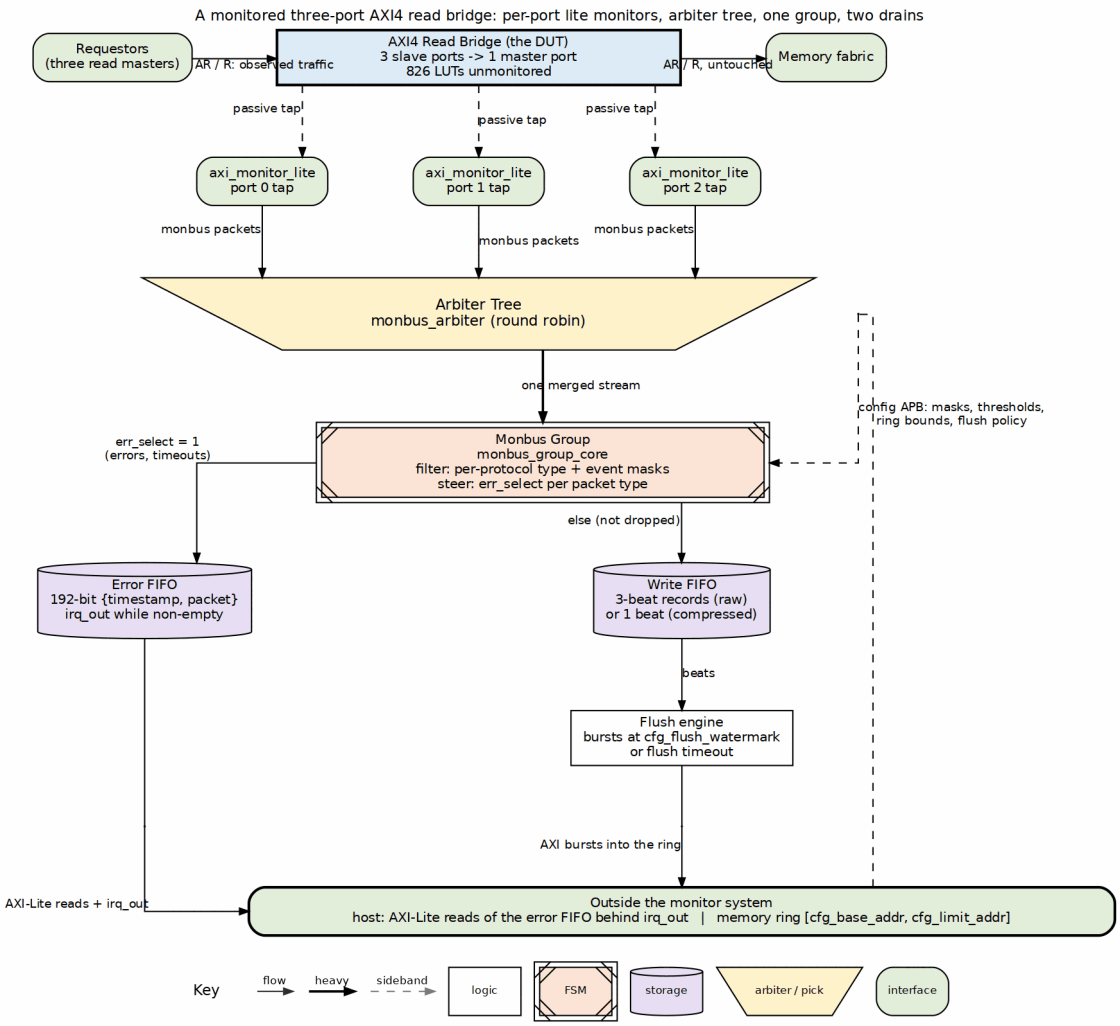
The fabric is sliced into five purpose planes. Each is a unit you can keep or drop at build time; dropping one never reroutes the others. Where you see `plane` in a name anywhere in this repository, this is the discipline it refers to.

Plane	Job	Representative members
Protocol	Observe the bus, packetize protocol events	`axi_monitor_lite`, `axi_monitor_base`, `axis_monitor_lite`, `cxs_monitor_lite`, the transaction manager/CAM/timeout/addr-check blocks
Error	Detect and packetize errors, timeouts, threshold crossings	`axi_monitor_reporter_error`, `..._timeout`, `..._threshold`
Debug	Completion and debug-trace packets	`axi_monitor_reporter_debug`, `..._compl`
Perf	Roll counters up into performance packets	`axi_monitor_reporter_perf`, in-datapath `axi_bus_meter`
Output	Arbitrate, filter, compress, drain packets to the host or memory	`monbus_arbiter` tree, `monbus_group_*` family, compressor, halfbeat packer, capture CAMs

The contract between planes is fixed and small: the protocol plane taps the bus passively and never drives it back; the error, debug and perf planes each consume only the events they own; every plane emits the same typed 128-bit `monitor_packet_t` with a 64-bit timestamp; and the output plane merges everything onto one monbus and drains it — an interrupt-backed error FIFO the host reads over AXI-Lite, and a write FIFO flushed as AXI bursts into a memory ring. The full membership table and the build personas (bring-up, silicon-perf, field-debug, full) live in the Ocelus plane map.

2.2 An implementation, end to end

The figure below is a real one — the three-port AXI4 read bridge fixture the cost numbers in chapter 10 come from, monitored the way the bridge generator monitors every build: one lite monitor per slave port, a round-robin arbiter tree, one shared group, two drains. The whole monitored bridge costs about 5,300 LUTs against 826 unmonitored, and meets timing on both boards with margin to spare.



A monitored three-port read bridge: per-port lite monitors, arbiter tree, group, error and trace drains

Three flows, traceable at a glance: the **observed traffic** crosses the top on the bridge's own AXI path — the monitors tap it passively (dashed) and never touch it. The **monitor packets** flow top-down the center from each tap through the arbiter tree into the group, which filters by packet type and steers by class. The **drains** leave through the boundary bar at the bottom: error-class records to the interrupt-backed FIFO the host reads over AXI-Lite, everything else as AXI bursts

into the memory ring. The configuration APB enters the group from the side; it owns every mask and threshold named in the following chapters.

2.3 What the integrator owns

Everything on the figure that is not fixed by the packet format is a knob, and each knob has a chapter:

Axis	Chapter
The spine itself: ports, tree, group, two drains	2
Which monitor: the lite, unless you can say why not	3
Identity space allocation (`unit_id` / `agent_id`)	4
Where to insert monitoring (port, mid-fabric, root)	5
Timestamp policy	6
Drain path selection	7
Packet-type filtering	8
Aggregation topology	9
What it costs	10
Validating a tweak in simulation	11
Relationship to Nexus (IEEE-ISTO 5001)	13

All citations are collected in the references chapter at the end.

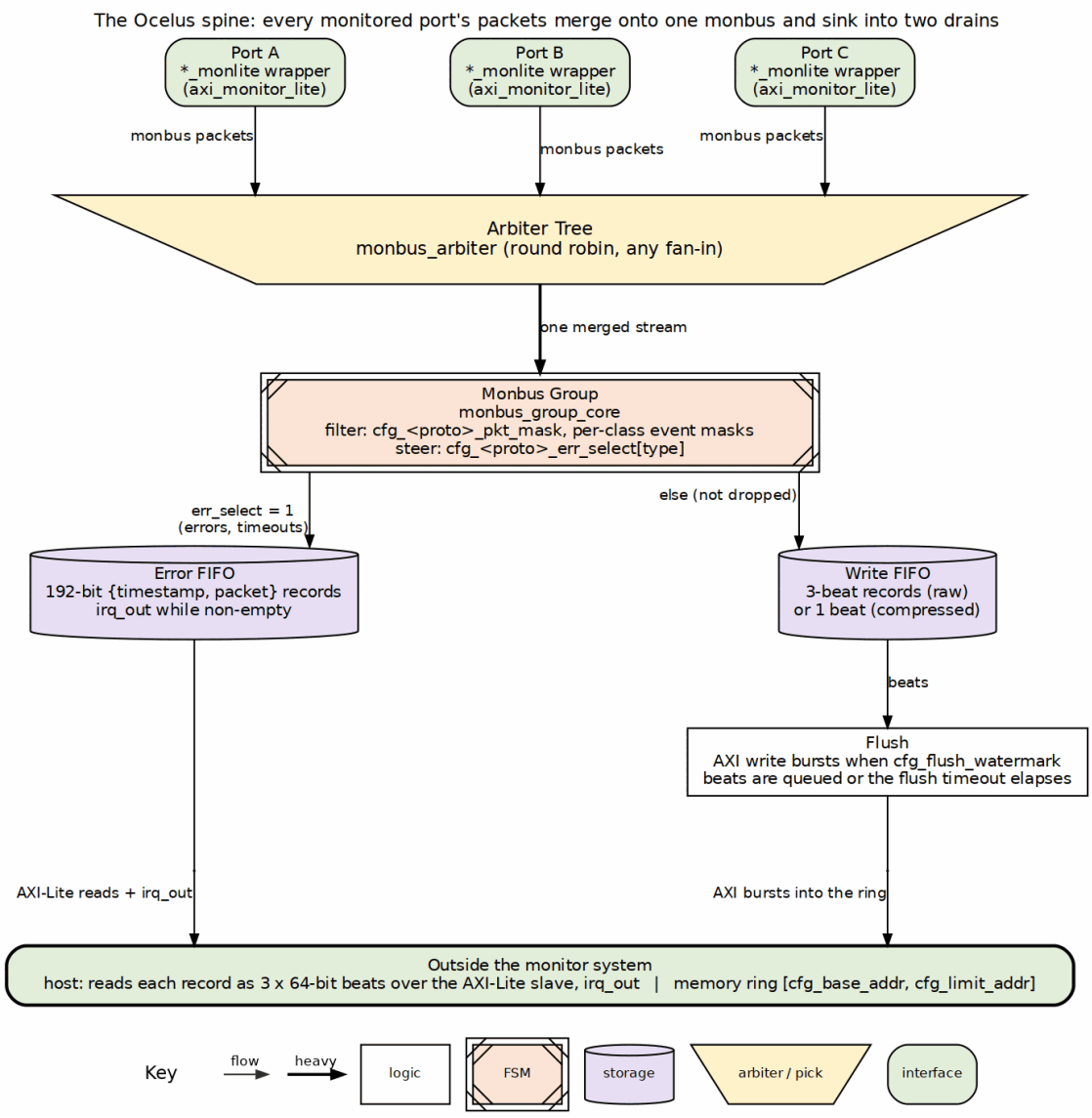
3 The spine

Every monitored port emits 128-bit packets onto a monitor bus (monbus). An arbiter tree merges the ports of a block into one stream, and a monbus group sinks that stream into two paths: an error FIFO the host reads over AXI-Lite behind an interrupt, and a write FIFO the group flushes as AXI bursts into a memory ring. Filtering happens at the group, per protocol and per packet type. The packet layout is fixed (monitor_package_spec.md); everything else on this page is a knob.

For the opening incident, the memory ring is the important sink. Steering the bridge, DMA, and stream error records there gives software one evidence area to inspect. Using each record's timestamp and source identity, it reconstructs the decode error followed by the two timeouts. This is more useful after a failure than a set of independent status bits that says only that each block eventually objected.

Two kinds of order matter. The ring's physical order is the order in which the group arbitrated and accepted records; delivery delays can place a later event ahead of an earlier one from another source. The timestamp records when the event was observed. With the proposed global usync time base, software can compare and merge records from different groups in event-time order at microsecond resolution. Ocelus keeps both orders rather than pretending that arrival at a drain is always the order in which events occurred.

> **Plane framing:** The spine is the output plane in action — `monbus_arbiter` merges streams from every protocol-plane port, and the `monbus_group` family filters and drains them off-chip. The ports themselves live in the protocol plane; everything after emission is the output plane. See Ocelus plane map.



Monitor system spine: ports, arbiter tree, group, two drains

4 Which monitor: the lite, unless you can say why not

Two monitors implement the port end of the spine. `axi_monitor_base`, the full monitor, tracks transactions in a CAM and drives six reporter cones (error, timeout, completion, threshold, performance, debug) with ID and address filtering. `axi_monitor_lite` tracks the same transactions in per-ID linked lists, emits errors, timeouts, completions with latency, thresholds and address-range events, and drops the performance and debug classes and the ID filter. Measured at their default parameters, out of context, on the Kintex-7 the boards ship on (monitor/monitor_characterization.md):

	Full monitor	Lite	Ratio
AXI4 read monitor, LUTs	7,006	1,115	6.3x
AXI4 read monitor, flops	5,219	1,027	5.1x
AXI-Lite read monitor, LUTs	2,891	929	3.1x
6.667 ns on Kintex-7 325T -2, register to register	misses (CAM at 16 slots)	meets, 0.8 to 1.5 ns spare	
10 ns on Artix-7 100T -1	misses by 1.45 ns	meets by 0.6 ns	

That is why the lite is the default: the bridge generator instantiates it on every monitored port, STREAM's three in-core monitors and RAPIDS' descriptor and AXIS monitors are lite, and every board build since 2026-09-26 carries it. It is also the more honest of the two under load. A full monitor that runs out of table entries stalls the command channel (`block_ready`); the lite never stalls the bus, counts the command it could not track (`refused_count`), and when its output queue is full it counts the events it could not deliver and reports the count in one `Error/EVENT_DROPPED` packet as soon as the bus is free. Its contract is exact and is proved in the tree: every command is completed, refused or live, and every generated event is delivered, reported dropped or pending, with no fourth outcome (`val/amba/test_axi_mon_block_ready.py`, `test_axi_monitor_soak.py`, `formal/amba/axi_monitor_lite`). A stalled bus is the one effect an instrument must never have on the design it watches.

What you give up, and when it matters:

Full-monitor feature	Lite	Reach for the full monitor if
performance windows (`cfg_perf_enable`, byte/beat/idle counters)	none; the in-datapath `axi_bus_meter` and the pass-through observer measure throughput without a monitor	you need windowed bandwidth numbers from the monitor itself
debug trace packets	none	you need per-beat trace, not events
ID-range filter (`ID_FILTER_ENABLE`)	none; one lite per port, sliced by `unit_id`/`agent_id` instead	several instances must share one port by ID
16 slots	8 by default, parameter	more than eight transactions in flight per port and every one must be tracked, not counted as refused
address filter	address ranges kept (`N_ADDR_RANGES`, error or match per range)	

The rest of this paper assumes the lite at the ports. Every axis below is the same for both monitors; the numbers are the lite's.

> **Plane framing:** Choosing the full monitor or the lite is a protocol-plane decision. The lite drops the perf and debug cones that belong to the perf and debug planes, but it still emits protocol-plane events for the error and debug planes downstream. See Ocelus plane map.

5 Identity space allocation

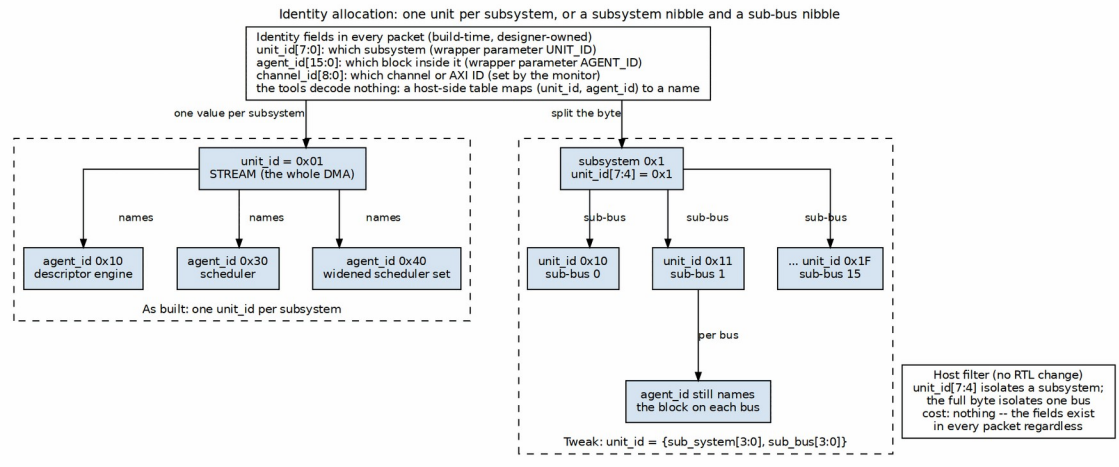
> **Plane framing:** Identity fields are assigned where the protocol plane produces the packet and then carried unchanged through every downstream plane. Allocating `unit_id` and `agent_id` is therefore a protocol-plane policy that every error, debug, perf and output-plane consumer inherits. See Ocelus plane map.

Three fields in every packet are designer-owned and fixed at build time:

Field	Bits	Set by	Meaning
`unit_id`	8	wrapper parameter `UNIT_ID`	which subsystem
`agent_id`	16	wrapper parameter `AGENT_ID`	which block inside it
`channel_id`	9	the monitor, from the transaction	which channel or AXI ID

The tools decode nothing from these fields; a host-side table maps the `(unit_id, agent_id)` tuple to a name. That makes the allocation yours. STREAM uses `unit_id = 0x01` for the whole DMA and spends `agent_id` on blocks: `0x10` descriptor engine, `0x30` scheduler, `0x40` the widened scheduler set (`projects/components/dma-ip/stream/rtl/macro/scheduler_group.sv`).

Worked example, one level down. A subsystem with several internal buses can split `unit_id` into a subsystem nibble and a sub-bus nibble: `unit_id = {sub_system[3:0], sub_bus[3:0]}` gives sixteen subsystems each with up to sixteen tracked internal buses, and `agent_id` is still free to name the block on each bus. A host filter on `unit_id[7:4]` then isolates a subsystem, and on the full byte a single bus, with no change to the RTL. The cost is nothing: the fields exist in every packet regardless.



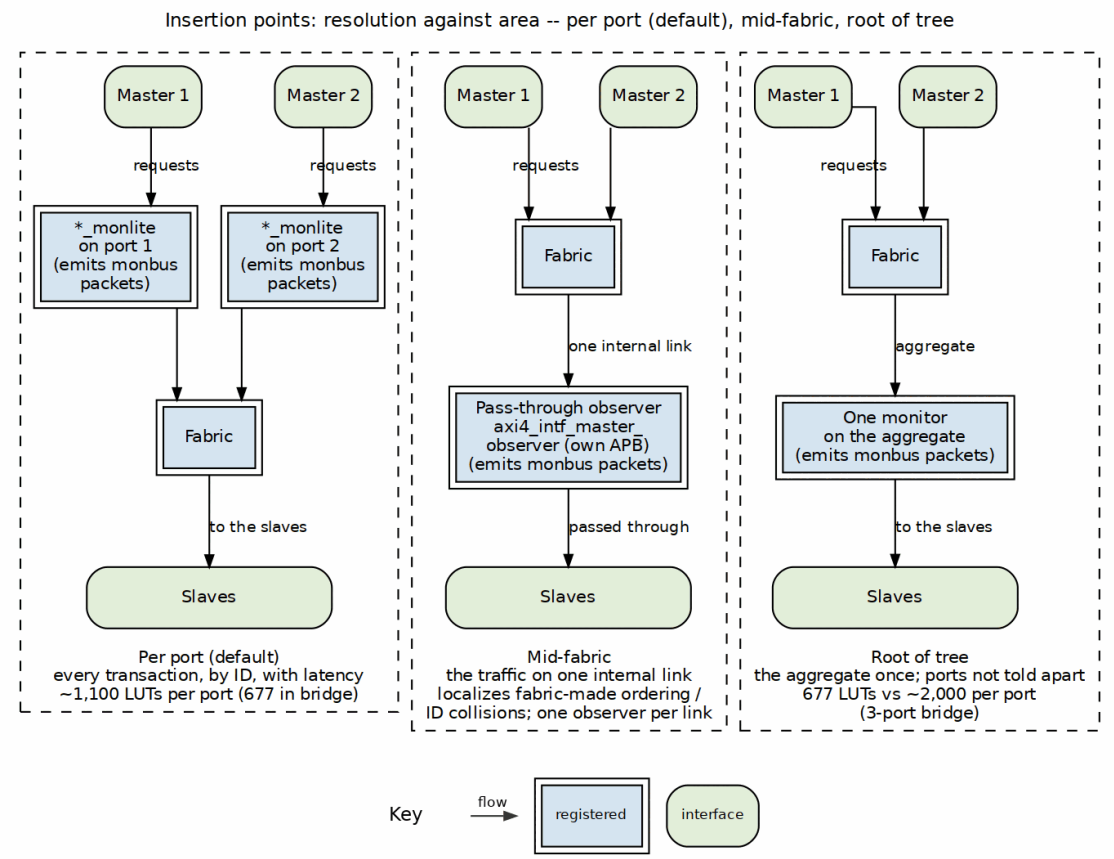
Identity allocation: one unit per subsystem, or a subsystem nibble and a sub-bus nibble

6 Where to insert monitoring

Plane framing: Insertion point is a protocol-plane decision — it determines where the bus is tapped and which `agent_id` the packet carries. The output plane (arbitration, drain, capture) is unaffected by whether the monitor sits on a port, a fabric link or a root. See Ocelus plane map.

Placement	What it sees	Cost	Use when
Per port (the default; the bridge generator puts a lite monitor on every monitored port)	every transaction on that port, attributed by ID, with completion latency	one lite per port: about 1,100 LUTs / 1,030 FFs at defaults, 677 / 831 with the bridge's narrower parameters (monitor_characterization.md, axi_monitor_lite.md)	you need to know WHICH port misbehaved
Mid-fabric	the traffic crossing one internal link, through a pass-through observer with its own APB configuration (`projects/components/utility-ip/misc/rtl/axi4_intf_master_observer.sv`)	one observer per link	localizing violations the fabric itself introduces (ordering, ID collisions between ports)
Root of tree	the aggregate of everything below, once	one monitor	area-constrained; you only need "something is wrong", not where

Per port and root of tree are the ends of one trade: resolution against area. A three-port read bridge with per-port lite monitors spends about 2,000 LUTs on monitors and about 1,400 on the shared group; the same bridge monitored once at its root spends 677 and 1,400, and can no longer tell the ports apart. With the full monitor the same per-port choice would cost about 21,000 LUTs, which is why per-port monitoring was not realistic before the lite and is the default with it.



Insertion points: per port, mid-fabric, root of tree

7 Timestamp policy

An ordered error history is only as useful as its clock. If the bridge, DMA, and stream in the opening incident each count from a different reset, their timestamps cannot establish which fault came first. Software is left with three local sequences and no trustworthy way to interleave them.

A 64-bit timestamp rides beside every packet (`monbus_timestamp_t`). Today it is the monbus group family's local free-running counter (`mon_time_out`), distributed to the wrappers as `i_mon_time`, and the group writes its low 60 bits into the first beat of every three-beat record. Within one group every packet is on one clock; across groups there is no shared time base.

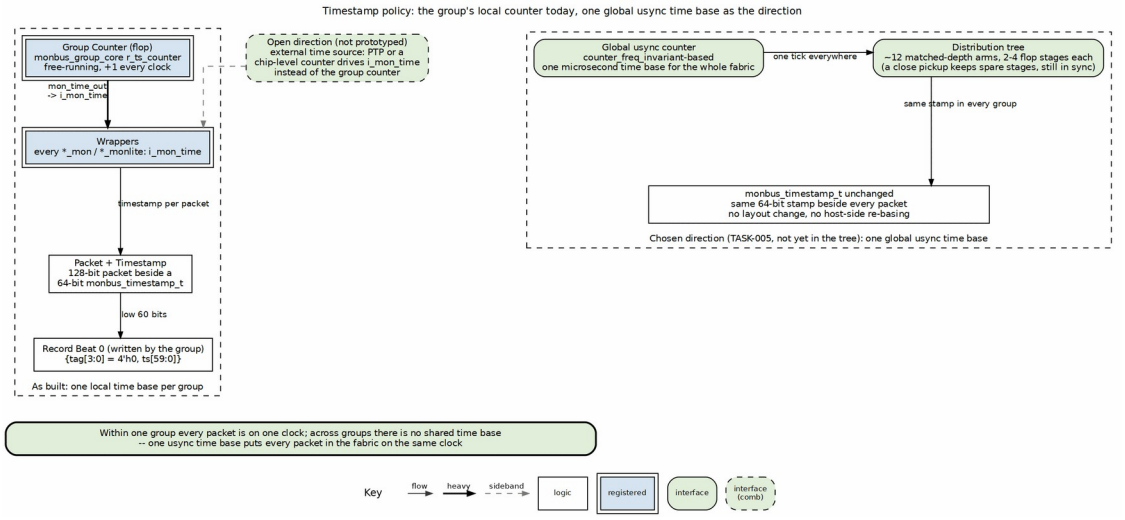
Two directions are open to the integrator:

- **Global usync time base** (the chosen direction, rtl/amba/monitors TASK-005): one system-wide microsecond counter — built on `counter_freq_invariant` so the tick rate survives clock-domain changes — sourced centrally and distributed on matched-depth arms (about twelve arms, two to four flop stages each; a consumer picked up nearby simply keeps spare stages in sync with the rest). Every monitor across the whole fabric — AMBA ports, STREAM, RAPIDS, bridge, delta — stamps packets from the same time base. Software can then merge error records from independent groups into one SoC-wide history without host-side re-basing. The local group counter stays as a parameter fallback.
- **External time source** (PTP or a chip-level counter): drive `i_mon_time` from it instead of the group's counter. The wrappers do not care where the value comes from.

Global time orders events with different ticks at microsecond resolution; two events in the same tick remain tied. It also does not make drain order equal to event order: arbitration and backpressure can delay a record after it was stamped. Software should merge per-group histories by timestamp, use source identity or another explicit policy for ties, and treat the resulting sequence as evidence rather than proof of causality.

Neither global usync nor an external-time integration is in the tree yet. The usync form is the one this paper recommends because it needs no new field, no host-side re-basing, and one distribution tree serves every plane.

> **Plane framing:** The timestamp policy is set at the output plane's `monbus_group` family today and stamped onto every packet that has already passed through the protocol, error, debug and perf planes. A change here reaches every plane, so keep the time base consistent across a build. See Ocelus plane map.



Timestamp policy: the group's local counter today, one global usync time base as the direction

8 Drain path selection

An interrupt answers, “What needs attention now?” A memory history answers, “What happened, and in what order?” An SoC needs both questions answered, but not necessarily for the same packet types.

The group has two sinks and a per-type steering mask (monbus_group.md):

Sink	Record	Reached by	Best for
Error FIFO	192-bit `{timestamp, packet}`, read back as three 64-bit beats over the AXI-Lite slave; `irq_out` while non-empty	`cfg_<proto>_err_select[type] = 1`	the few packets a handler must act on now: errors, timeouts
Write FIFO	the same record as three beats (raw) or one beat (compressed), flushed as AXI bursts into the ring `[cfg_base_addr, cfg_limit_addr]` when `cfg_flush_watermark` beats are queued or the flush timeout elapses	everything not dropped and not selected for the error FIFO	bulk trace and memory-resident histories: errors, completions, latencies, thresholds

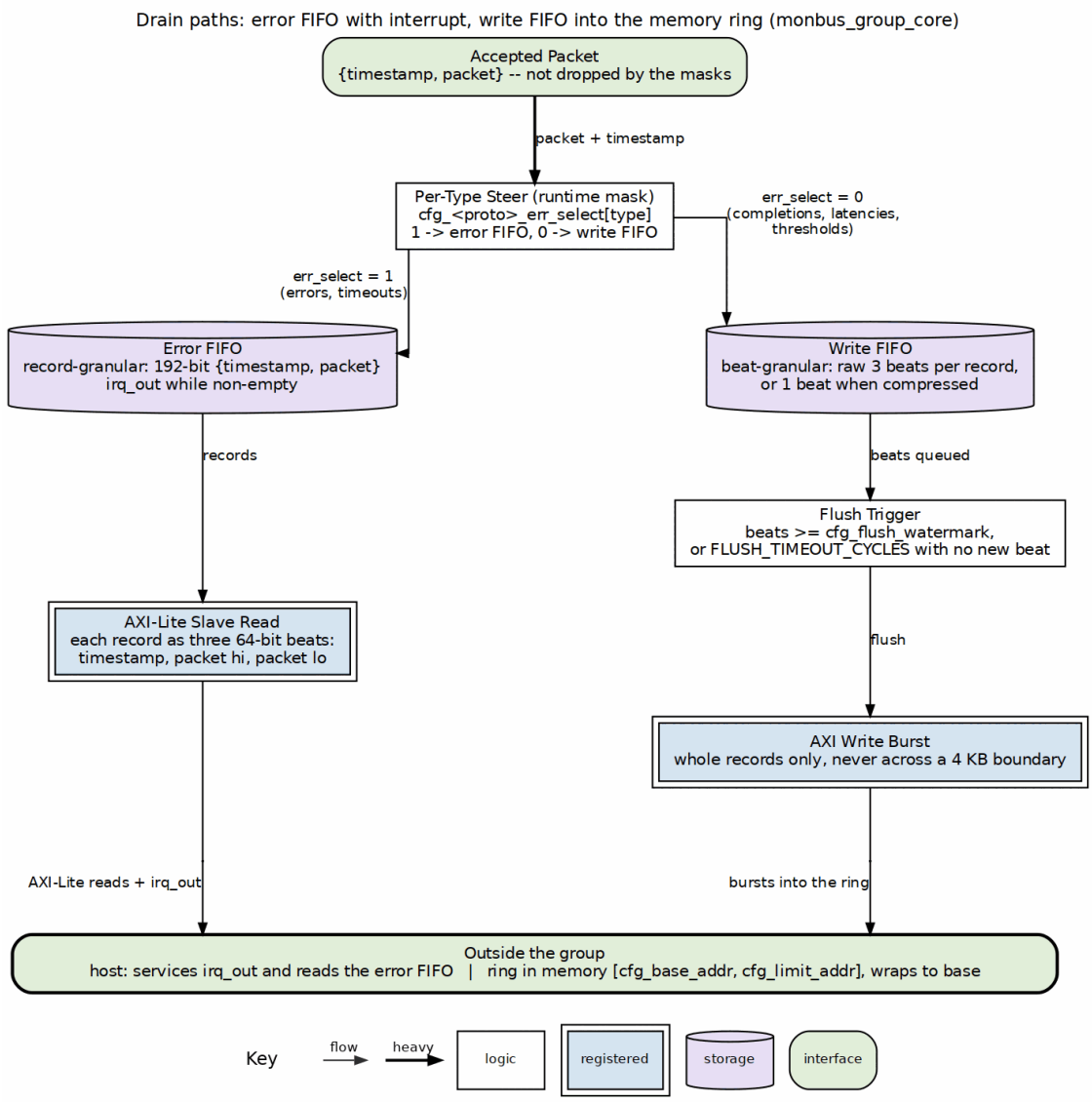
The choice is per packet type per protocol, at runtime. The common shape is errors and timeouts to the interrupt path and everything else to the trace ring; a debug session flips completions over to the error FIFO for a while and reads them one at a time.

For the opening incident, the integrator instead steers the error classes of interest to the memory ring. The host sorts the records by event timestamp and sees the bridge decode error first, then the DMA and stream timeouts, rather than collecting fragments from three handlers. The raw ring remains in drain order; its timestamps provide the chronological view. The ring is circular, so its useful history is bounded by its size and by flush and loss policy; it is not an infinite or inherently

nonvolatile log. A system that needs it after reset must preserve or capture the memory contents as part of its reset architecture.

Steering selects a destination; it does not duplicate a packet into both sinks. Urgent classes can therefore use the interrupt FIFO while archival classes use memory. With multiple groups, each ring preserves its own acceptance order, and the proposed global usync timestamps let software merge the records into an SoC-wide event-time history.

> **Plane framing:** Drain-path selection is an output-plane decision: `monbus_group_core` steers each packet type to the interrupt/read FIFO or the master-write ring. The protocol and reporter planes only emit typed packets; they do not know which drain will consume them. See Ocelus plane map.



Drain paths: error FIFO with interrupt, write FIFO into the memory ring

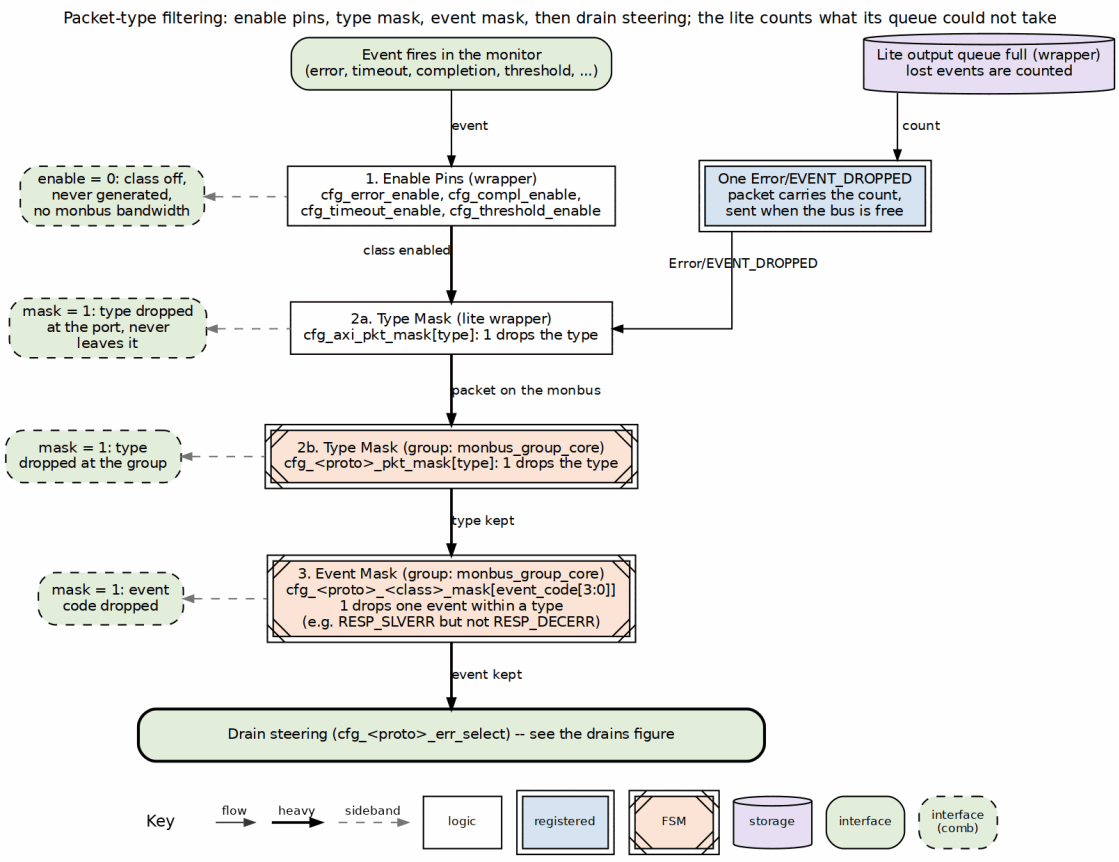
9 Packet-type filtering

Three layers, all runtime-programmable over the control APB:

1. **Type mask** at the group, `cfg_<proto>_pkt_mask[type]`: 1 drops the type. Also on every lite wrapper (`cfg_axi_pkt_mask`) so a dropped type never even leaves the port.
2. **Event mask** at the group, `cfg_<proto>_<class>_mask[event_code[3:0]]`: 1 drops one event within a type, for example `RESP_SLVERR` but not `RESP_DECERR`.
3. **Enable pins** on the wrappers, `cfg_error_enable`, `cfg_compl_enable`, `cfg_timeout_enable`, `cfg_threshold_enable`: a class that is off is not generated at all, so it costs no monbus bandwidth.

The congestion pitfall, and how the lite retires it. On the full monitor, completion and performance packets are the high-rate classes; with both enabled on a busy port the monbus saturates and lower-priority packets are lost silently. The rule from AXI_Monitor_Configuration_Guide.md stands there: never enable `cfg_compl_enable` and `cfg_perf_enable` together. The lite has no performance class to collide with, holds a timeout or a latency event that loses the pick until it can go, and when its queue is genuinely full it counts the loss and reports the count in one `Error/EVENT_DROPPED` packet as soon as the bus is free. Nothing is lost silently; the consumer always knows how many events it did not see, which is the property a filtering strategy can be built on.

> **Plane framing:** Filtering happens in two planes. The wrapper-level enable pins (`cfg_error_enable`, `cfg_compl_enable`, …) are protocol-plane knobs that stop packets from being generated; the group-level type and event masks are output-plane knobs that drop already-generated packets. Use both, but prefer stopping at the source. See Ocelus plane map.



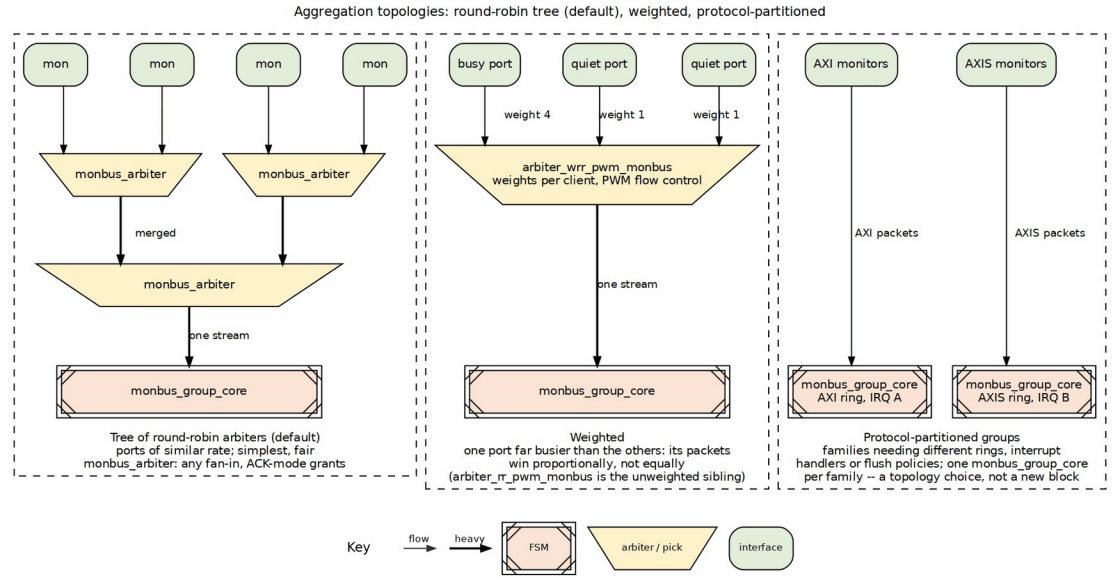
Packet-type filtering: enable pins, type mask, event mask, then drain steering; the lite counts what its queue could not take

10 Aggregation topology

Plane framing: Aggregation topology is an output-plane decision — it determines how protocol-plane sources are merged before they reach the `monbus_group` drain. The packets themselves are unchanged by the tree shape. See Ocelus plane map.

Topology	Block	When
Tree of round-robin arbiters (default)	`monbus_arbiter`, any fan-in, ACK-mode grants	ports of similar rate; simplest, fair
Weighted	`arbiter_wrr_pwm_monbus` (weights per client, PWM flow control); `arbiter_rr_pwm_monbus` is the unweighted sibling	one port far busier than the others and you want its packets to win proportionally, not equally
Protocol-partitioned groups	one `monbus_group_core` per protocol family instead of the shared AXI / AXIS / CORE slots	the families need different rings, different interrupt handlers, or different flush policies

The group core is shared by every wrapper family (AXI4, AXI5, AXI-Lite, AXIS, Wishbone), so partitioning is a topology decision, not a new block.



Aggregation topologies: round-robin tree, weighted, protocol-partitioned

11 What it costs

Standalone, at default parameters, from the characterization sweep (monitor/monitor_characterization.md), and in place, from the bridge fixture with the bridge's parameters (axi_monitor_lite.md):

	Full monitor	Lite
One AXI4 read monitor at defaults (16 / 8 slots), LUTs / FFs	7,006 / 5,219	1,115 / 1,027
One read monitor in the bridge, LUTs / FFs	3,249 / 1,628	677 / 831
Three-port read bridge with group, LUTs	12,625	5,339
The unmonitored bridge, LUTs	826	826
Kintex-7 325T -2 at 6.667 ns, bridge WNS	+0.212 ns	+1.092 ns
Kintex-7 at 6.667 ns, standalone at defaults, register to register	-0.166 ns	+1.005 ns
A group (error FIFO 64 records, write FIFO 96 beats), LUTs / FFs	1,700 to 2,000 / 1,150 to 1,390	same, shared

The lite bridge fixture meets 10 ns on the Artix-7 100T -1 since 2026-09-28 (+0.366 ns), after the group planner and the lite's event stage were re-pipelined (amba ISSUE-001, rtl/amba/monitors ISSUE-002). The per-variant characterization – every wrapper family, with and without clock gating, full against lite, on both parts, from one repeatable flow – is monitor/monitor_characterization.md. Its headline: at default parameters the full AXI monitor is about 7,000 LUTs and the lite about 1,100, and the lite meets 150 MHz on the Kintex-7 with 0.8 ns or more to spare where the 16-slot full monitor does not.

Plane framing: Cost denominators are plane-shaped. The protocol-plane monitor choice (full vs lite) dominates per-port area; the output-plane group, compression and CAM options dominate the shared drain cost; and the error, debug and perf reporter cones are paid for only when their `ENABLE_*_LOGIC` parameters are set. See Ocelus plane map.

12 Validating a tweak in simulation

Three templates in the tree exercise exactly the knobs above:

- `val/amba/test_axi_mon_block_ready.py`: saturate a port and prove the monitor's admission contract, `block_ready` on the full monitor and the refuse-count identity on the lite.
- `val/amba/test_axi_monitor_soak.py::monitor_soak_monlite`: random traffic with backpressure, asserting that every generated event is delivered or counted as dropped. Change a mask or a drain and this is the identity that must still close.
- `formal/amba/monbus_group_core`: the routing rule, `monbus_ready`, the FIFO accounting and the legality of the flush burst, proved at the ports. A change to the group's filtering or drain logic re-runs against it.

The error-injection template the original outline named (`test_bridge_1x2_rd_monitor_error_inject.py`) is not in the tree; the bridge's generated monitor stress tests cover that ground.

13 Out of scope

Packet bit layout (monitor_package_spec.md), per-module ports and timing (monitor/), and test recipes (the test sources).

14 Relationship to Nexus (IEEE-ISTO 5001)

The references chapter already names the Nexus 5001 Forum standard (the 2003 Version 2.0 text, local PDF `rtl/amba/references/DebugTraceInterfaceStandard__IeeeIsto2003.pdf`). This chapter is that reference expanded: why the comparison keeps coming up, where the correspondence is real, and where the two systems split.

Nexus standardizes how an embedded processor reports its own activity to an external debug tool: typed messages (each led by a 6-bit TCODE), per-source IDs, optional timestamps, and program / data / ownership trace, streamed off-chip through a dedicated auxiliary port while the core runs. Ocelus looks similar from a distance – typed packets, per-source identity, timestamps – and the correspondence is real enough that four of the six integrator axes in this paper have direct Nexus counterparts:

Nexus mechanism	Ocelus counterpart	Chapter
Source IDs in each message	Identity space allocation	4
TSTAMP field per message	Timestamp policy	6
Message typing (6-bit TCODE)	Packet-type field, filtered per type at the group	8
AUX trace port (MDO/MSEO pins)	Drain path: error FIFO and memory ring	7

That is why the comparison keeps coming up. The split is everything the table leaves out.

Nexus answers "what happened?" Ocelus answers "is something wrong right now?" Nexus is observability: enough state leaves the chip to reconstruct processor behavior after the fact. Ocelus is operational monitoring: it reports condition crossings on live AMBA traffic and hands them to software through the drains. A flight recorder versus a smoke alarm.

Three structural differences follow:

1. **Consumer location.** Nexus streams to an external debugger over the AUX port; every message is target-to-tool. Ocelus packets terminate on-chip – the error FIFO the host reads over AXI-Lite behind an interrupt, and the memory ring the group fills with AXI bursts. (The planned HIVE policy tier reads the same drains.)
2. **Information policy.** Nexus pays pins, serialization logic, and external capture hardware to stream continuously. Ocelus emits edge-triggered packets only, which is what makes the 0.9 posture affordable: lite monitor on every port, full monitor only where justified (chapter 10).

3. **Loop closure.** Nexus is open loop: data leaves and nothing in the system changes. Ocelus exists to close one: detect on the fabric, drain, and let software escalate.

The watchpoint distinction. The crispest single-sentence version: what Nexus uses as a filter, Ocelus uses as the output. In Nexus, a watchpoint does not halt the core – it emits a message to external tools and, in Class 4 implementations, gates the start of program and data trace. The watchpoint exists to control what gets traced. In Ocelus, the condition crossing is the product: the monitor raises a packet, the group filters it, the drain carries it. There is no trace stream for it to gate.

Placement. Nexus instruments the processor core; Ocelus instruments the bus. That is the insertion-point axis (chapter 5), and it is why the two are complements, not competitors: on an SoC running RISC-V, the cores can speak N-Trace (a strict subset of IEEE-5001 Nexus messaging) while Ocelus covers the fabric around them. Nexus-family trace watches the processors; Ocelus watches the system they run on.

Version note. The repository holds the 2003 Version 2.0 text cited in chapter 1. IEEE-ISTO 5001-2012 is the later revision, and the Nexus 5001 Forum dissolved in September 2020 with the standard left openly available – none of which changes the comparison, since the mechanisms above are common to both revisions.

15 References

Ocelus is an original design of this repository. The papers and standards below are related work for comparison, not sources of the design; protocol specifications define the rules the monitors observe. Where a local PDF exists it is indexed in `rtl/amba/references/README.md`; the handbook's citation convention keeps the papers in that central store and the per-chapter usage notes here.

15.1 Interconnect monitoring (the closest relatives)

- Ciordas, C., Basten, T., Rădulescu, A., Goossens, K., van Meerbergen, J. “An Event-Based Monitoring Service for Networks on Chip.” *ACM Trans. Design Automation of Electronic Systems*, 10(4):702-723, Oct. 2005. DOI: 10.1145/1109118.1109126 — probes, events and monitoring traffic on the interconnect; reconfigurable probes with a run-time programming model; probe area at 17-24% of NoC area, the number Ocelus’ lite posture undercuts (chapters 2, 8, 10).
- Ciordas, C., Goossens, K., Basten, T., Rădulescu, A., Boon, A. “Transaction Monitoring in Networks on Chip: The On-Chip Run-Time Perspective.” *Proc. IEEE Int. Symp. Industrial Embedded Systems (IES 2006)*, pp. 1-10, Oct. 2006. DOI: 10.1109/IES.2006.357464 — transaction monitors attached to interconnect links; transaction monitoring at about 5% of NoC area (chapters 5, 10).
- Goossens, K., Vermeulen, B., van Steeden, R., Bennebroek, M. “Transaction-Based Communication-Centric Debug.” *Proc. 1st Int. Symp. Networks-on-Chip (NOCS 2007)*, pp. 95-106, May 2007. DOI: 10.1109/NOCS.2007.46 — monitors plus a dedicated event-distribution interconnect; monitor placement for communication-centric debug; monitors plus event interconnect at 4.5% of NoC area (chapters 2, 5, 10).
- Fiorin, L., Palermo, G., Silvano, C. “MPSoCs Run-Time Monitoring through Networks-on-Chip.” *Proc. Design, Automation & Test in Europe (DATE 2009)*, pp. 558-561, Apr. 2009. DOI: 10.1109/DATE.2009.5090726 — probes in the network interfaces feeding a central collector (chapters 2, 5).

15.2 Debug and trace standards

- IEEE-ISTO 5001-2003. *The Nexus 5001 Forum Standard for a Global Embedded Processor Debug Interface*, Version 2.0, IEEE Industry Standards and Technology Organization, Dec. 2003 (defining non-peer standard) — a message-based trace standard (typed messages, source IDs, timestamps): §5.2 SRC field ~ `unit_id`/`agent_id` (chapter 4), §4.11 absolute vs relative timestamps (chapter 6), §4.4.3/§4.5.3 lost trace reported rather than stalling ~ the lite's refuse and drop counts (chapters 3, 8). Local PDF: `rtl/amba/references/DebugTraceInterfaceStandard__IeeeIsto2003.pdf`.
- IEEE-ISTO 5001-2012. *The Nexus 5001 Forum Standard Embedded Processor Debug Interfaces*, Version 3.0, Apr. 2012 — later revision; watchpoint- gated trace is its Class 4 compliance level. Forum dissolved Sept. 2020 with the standard left openly available (chapter 13).
- RISC-V International. *N-Trace Specification* (riscv-nexus-trace) — instruction trace for RISC-V cores; a strict subset of IEEE-5001 Nexus messaging, and the complement to Ocelus on a RISC-V SoC: N-Trace watches the processors, Ocelus watches the system they run on (chapter 13).

15.3 Debug-infrastructure surveys and tracers

- Hopkins, A.B.T., McDonald-Maier, K.D. “Debug Support for Complex Systems On-Chip: A Review.” *IEE Proc. Computers and Digital Techniques*, 153(4):197-207, 2006. DOI: 10.1049/ip-cdt:20050194 — on-chip trace buffers versus trace ports (chapter 7).
- Abramovici, M., Bradley, P., Dwarakanath, K., Levin, P., Memmi, G., Miller, D. “A Reconfigurable Design-for-Debug Infrastructure for SoCs.” *Proc. 43rd Design Automation Conf. (DAC 2006)*, pp. 7-12, July 2006. DOI: 10.1145/1146909.1146916 — programmable triggers in a debug fabric (chapter 8).
- Yang, F.-C., Lin, Y., Kao, C.-F., Huang, I.-J. “An On-Chip AHB Bus Tracer With Real-Time Compression and Dynamic Multiresolution Supports for SoC.” *IEEE Trans. VLSI Systems*, 19(4):571-584, Apr. 2011. DOI: 10.1109/TVLSI.2009.2039811 — a bus tracer recording into an on-chip circular buffer (chapter 7).

15.4 Time and ordering

- Lamport, L. “Time, Clocks, and the Ordering of Events in a Distributed System.” *Communications of the ACM*, 21(7):558-565, July 1978. DOI: 10.1145/359545.359563 — ordering events across groups that share no clock (chapter 6). Local PDF: `rtl/amba/references/LogicalClocksEventOrdering__Lamport1978.pdf`.
- IEEE Std 1588-2008. *IEEE Standard for a Precision Clock Synchronization Protocol for Networked Measurement and Control Systems*, July 2008. DOI: 10.1109/IEEESTD.2008.4579760 — the precision time protocol named as an external time source (chapter 6).

15.5 Structures the design converges with

- Tamir, Y., Frazier, G.L. “High-Performance Multi-Queue Buffers for VLSI Communication Switches.” *Proc. 15th Annual Int. Symp. Computer Architecture (ISCA 1988)*, pp. 343-354, May 1988. DOI: 10.1109/ISCA.1988.5245 — a comparable structure to the lite monitor’s per-ID linked lists in a shared slot pool (chapter 3). Local PDF: `rtl/amba/references/LinkedListMultiQueueBuffer__Tamir1988.pdf`.
- Pagiamtzis, K., Sheikholeslami, A. “Content-Addressable Memory (CAM) Circuits and Architectures: A Tutorial and Survey.” *IEEE J. Solid-State Circuits*, 41(3):712-727, Mar. 2006. DOI: 10.1109/JSSC.2005.864128 — the CAM behind the full monitor’s table (chapter 3). Local PDF: `rtl/common/references/ContentAddressableMemory__Pagiamtzis2006.pdf`.
- Gupta, P., McKeown, N. “Designing and Implementing a Fast Crossbar Scheduler.” *IEEE Micro*, 19(1):20-28, Jan./Feb. 1999. DOI: 10.1109/40.748793 — the round-robin arbiter core of the aggregation tree (chapter 9).
- Katevenis, M., Sidiropoulos, S., Courcoubetis, C. “Weighted Round-Robin Cell Multiplexing in a General-Purpose ATM Switch Chip.” *IEEE J. Selected Areas in Communications*, 9(8):1265-1279, Oct. 1991. DOI: 10.1109/49.105173 — interleaved weighted round robin, the weighted aggregation option (chapter 9).
- Dally, W.J., Towles, B. *Principles and Practices of Interconnection Networks*, Morgan Kaufmann, 2004, ch. 18 “Arbitration”, §18.4.2 and §18.4.4 — textbook treatment of the arbiter cores (secondary source) (chapter 9).

Back to Main Index | **rtl-amba Overview**

16 RTL AMBA Modules Index

This directory contains documentation for the AMBA (Advanced Microcontroller Bus Architecture) RTL library, providing comprehensive implementations of ARM AMBA bus protocols. The library supports both **AMBA 4** and **AMBA 5** specifications across APB, AXI, AXI-Lite, and AXI-Stream protocols.

16.1 Overview

- **Overview** - Complete overview of the RTL AMBA library architecture and protocol implementations
- **Ocelus White Paper** - Ocelus as a design surface: identity allocation, insertion points, timestamp policy, drain paths, filtering, aggregation topology, and what each costs. One chapter per file under `rtl-amba-monitors/whitepaper/`; branded PDF `docs/pdfs/Ocelus_White_Paper.pdf` via `generate_rtl_pdfs.sh monitor-wp`.

16.2 AMBA 4 Protocol Implementations

AMBA 4 modules provide the foundation for high-performance SoC interconnect, widely deployed in production systems. These modules are mature, fully verified, and optimized for synthesis.

16.2.1 WB4 (Wishbone B4, pipelined)

Open-standard peripheral bus; the pair speaks B4 pipelined mode behind the same command/response queues as the APB4 pair. There is no Wishbone B5.

Wishbone B4 Module Documentation

- **wb4_master** - Pipelined Wishbone master with command/response queues; credit-gated issue, RTY reported as status
- **wb4_slave** - Pipelined Wishbone slave with command/response queues; in-order registered termination, orphan-response guard
- **wb4_monitor** - Monitor on the wb4 queues emitting PROTOCOL_WB monitor bus packets; in-order tracking, timeouts, latency, address-range check
- **wb4_retry** - RTY retry block on the FUB side of the master; in-order completion buffer, budget and delay
- **wb4_master_retry** - wb4_retry + wb4_master with the master's ports
- **wb4_master_cg** / **wb4_slave_cg** - Clock-gated variants (amba_clock_gate_ctrl, runtime enable/idle count)
- **wb4_slave_cdc** - Slave with its queues crossed to another clock domain (two gaxi_fifo_async)

16.2.2 APB4 (Advanced Peripheral Bus)

Simple, low-power peripheral bus for control registers and low-bandwidth devices.

APB4 Module Documentation

16.2.2.1 Core Components

- **apb4_master** - APB master with command/response interfaces and FIFO buffering
- **apb4_master_cg** - Clock-gated APB master for power optimization
- **apb4_slave** - APB slave with configurable address decoding and response generation
- **apb4_slave_cg** - Clock-gated APB slave for power optimization
- **apb4_monitor** - APB protocol monitor for verification and debugging

16.2.2.2 Clock Domain Crossing

- **apb4_slave_cdc** - APB slave with clock domain crossing support
- **apb4_slave_cdc_cg** - Clock-gated CDC APB slave

16.2.2.3 Test and Development

- **apb4_master_stub** - APB master stub for testing and simulation
- **apb4_slave_stub** - APB slave stub for testing and simulation

---

16.2.3 AXI4-Full (Memory-Mapped Interface)

High-performance memory-mapped interface supporting burst transactions and multiple outstanding operations.

AXI4 Module Documentation

16.2.3.1 Master Components

- **axi4_master_rd** - AXI4 read master with skid buffers and burst support
- **axi4_master_rd_cg** - Clock-gated AXI4 read master
- **axi4_master_wr** - AXI4 write master with address/data coordination
- **axi4_master_wr_cg** - Clock-gated AXI4 write master

16.2.3.2 Slave Components

- **axi4_slave_rd** - AXI4 read slave with configurable response handling
- **axi4_slave_rd_cg** - Clock-gated AXI4 read slave
- **axi4_slave_wr** - AXI4 write slave with write response generation
- **axi4_slave_wr_cg** - Clock-gated AXI4 write slave

16.2.3.3 Test Stubs

- **axi4_master_stub** - Complete AXI4 master stub
- **axi4_master_rd_stub** - AXI4 read master stub
- **axi4_master_wr_stub** - AXI4 write master stub
- **axi4_slave_stub** - Complete AXI4 slave stub
- **axi4_slave_rd_stub** - AXI4 read slave stub
- **axi4_slave_wr_stub** - AXI4 write slave stub

16.2.4 AXI4-Lite (Register Interface)

Simplified AXI4 for register access with single outstanding transaction support.

AXI4-Lite Module Documentation

16.2.4.1 Master Components

- **axil4_master_rd** - AXI4-Lite read master for register access
- **axil4_master_rd_cg** - Clock-gated AXI4-Lite read master
- **axil4_master_wr** - AXI4-Lite write master for register access
- **axil4_master_wr_cg** - Clock-gated AXI4-Lite write master

16.2.4.2 Slave Components

- **axil4_slave_rd** - AXI4-Lite read slave with register interface
- **axil4_slave_rd_cg** - Clock-gated AXI4-Lite read slave
- **axil4_slave_wr** - AXI4-Lite write slave with register interface
- **axil4_slave_wr_cg** - Clock-gated AXI4-Lite write slave

16.2.5 AXI4-Stream (Streaming Interface)

High-throughput unidirectional streaming for data processing pipelines.

AXI4-Stream Module Documentation

16.2.5.1 Stream Components

- **axis4_master** - AXI4-Stream master for data streaming
- **axis4_master_cg** - Clock-gated AXI4-Stream master
- **axis4_slave** - AXI4-Stream slave for data reception
- **axis4_slave_cg** - Clock-gated AXI4-Stream slave

16.3 AMBA 5 Protocol Implementations

AMBA 5 modules provide enhanced features for next-generation SoC designs, including atomic operations, extended QoS, memory tagging, and improved power management signaling.

16.3.1 APB5 (Advanced Peripheral Bus - AMBA 5)

Enhanced APB with wake-up signaling, non-secure extension, and user signals.

APB5 Module Documentation

Feature	APB4	APB5
Protection	PPROT[2:0]	PPROT[2:0] + PNSE
Wake-up	Not supported	PWAKEUP signal
User Signals	Not supported	PAUSER, PWUSER, PRUSER, PBUSER

16.3.1.1 Core Components

- **apb5_master** - APB5 master with command/response interface
- **apb5_slave** - APB5 slave with buffered cmd/rsp interface
- **apb5_slave_cdc** - APB5 slave with clock domain crossing
- **apb5_monitor** - APB5 transaction monitor

16.3.1.2 Clock-Gated Variants

- **apb5_master_cg** - Clock-gated APB5 master
- **apb5_slave_cg** - Clock-gated APB5 slave
- **apb5_slave_cdc_cg** - Clock-gated CDC APB5 slave

16.3.1.3 Test Stubs

- **apb5_master_stub** - APB5 master stub for testing
- **apb5_slave_stub** - APB5 slave stub for testing

16.3.2 AXI5 (High-Performance Memory-Mapped - AMBA 5)

Enhanced AXI with atomic operations, memory tagging, and extended QoS.

AXI5 Module Documentation

Feature	AXI4	AXI5
Outstanding	Up to 16	Up to 256

Feature	AXI4	AXI5
Atomic Ops	Not supported	AtomicStore, AtomicLoad, AtomicSwap, AtomicCompare
Memory Tagging	Not supported	AWMEMATTR, ARMEMATTR
Data Poisoning	Not supported	RPOISON, WPOISON

16.3.2.1 Master Components

- **axi5_master_rd** - AXI5 read master with skid buffers
- **axi5_master_rd_cg** - Clock-gated AXI5 read master
- **axi5_master_wr** - AXI5 write master with address/data coordination
- **axi5_master_wr_cg** - Clock-gated AXI5 write master

16.3.2.2 Slave Components

- **axi5_slave_rd** - AXI5 read slave with configurable response
- **axi5_slave_rd_cg** - Clock-gated AXI5 read slave
- **axi5_slave_wr** - AXI5 write slave with response generation
- **axi5_slave_wr_cg** - Clock-gated AXI5 write slave

16.3.2.3 Integrated Monitor Modules

- **axi5_master_rd_mon** - Read master with integrated monitor
- **axi5_master_rd_mon_cg** - Clock-gated read master with monitor
- **axi5_master_wr_mon** - Write master with integrated monitor
- **axi5_master_wr_mon_cg** - Clock-gated write master with monitor
- **axi5_slave_rd_mon** - Read slave with integrated monitor
- **axi5_slave_rd_mon_cg** - Clock-gated read slave with monitor
- **axi5_slave_wr_mon** - Write slave with integrated monitor
- **axi5_slave_wr_mon_cg** - Clock-gated write slave with monitor

16.3.3 AXI5-Stream (Streaming Interface - AMBA 5)

Enhanced streaming with wake-up signaling and data poisoning support.

AXI5-Stream Module Documentation

Feature	AXI4-Stream	AXI5-Stream
Wake-up	Not supported	TWAKEUP signal
Data Poisoning	Not supported	TPOISON

16.3.3.1 Stream Components

- **axis5_master** - AXI5-Stream master for data streaming
- **axis5_master_cg** - Clock-gated AXI5-Stream master
- **axis5_slave** - AXI5-Stream slave for data reception
- **axis5_slave_cg** - Clock-gated AXI5-Stream slave

16.3.4 ACE (AXI Coherency Extensions)

AXI4 extended with coherent transaction types and snoop channels, for the `cache-ip` family (amber/jet caches, onyx CCU).

ACE Module Documentation

16.3.4.1 Front-Side Movers

- **axi4ace_master_rd** - ACE read master with `ARSNOOP[3:0]` and auto-pulsed `RACK`
- **axi4ace_master_wr** - ACE write master with `AWSNOOP[2:0]` and auto-pulsed `WACK`
- **axi4ace_slave_rd** - ACE read slave with `ARSNOOP[3:0]` pass-through
- **axi4ace_slave_wr** - ACE write slave with `AWSNOOP[2:0]` pass-through

16.3.4.2 Snoop-Side Movers

- **axi4ace_snoop_slave** - Cache-side snoop responder transport: AC in, CR/CD out
- **axi4ace_snoop_master** - CCU-side snoop initiator transport: AC out, CR/CD in

16.3.4.3 Monitor Wrappers

- All eight `axi4ace_*_monlite` wrappers are documented collectively in **axi_monitor_lite_wrappers**

16.3.5 CXS (Credited eXtensible Stream)

Credited point-to-point packet streaming for wide links (on-chip interconnect to a PCIe/CCIX/CXL controller). One family covers spec Issues A-D and CXS-Lite, selected by `CXS_ISSUE`.

CXS Module Documentation

- **cxs_master** - CXS transmitter: credit counter, link activation, check-signal generation
- **cxs_master_cg** - Clock-gated transmitter (gates in link STOP)
- **cxs_slave** - CXS receiver: the receive FIFO is the credit pool
- **cxs_slave_cg** - Clock-gated receiver (CXSACTIVEREQ wakes it)
- **cxs_master_monlite** / **cxs_slave_monlite** - the endpoints with the CXS lite Ocelus (cxs_monitor_lite)

16.4 Shared Infrastructure and Utilities

Infrastructure components used across all AMBA protocols.

16.4.1 Generic AXI (GAXI) Components

GAXI Module Documentation

- **gaxi_fifo_sync** - Synchronous FIFO for GAXI interfaces
- **gaxi_fifo_async** - Asynchronous FIFO for clock domain crossing
- **gaxi_skid_buffer** - Skid buffer for pipeline optimization
- **gaxi_skid_buffer_async** - Asynchronous skid buffer

16.4.2 Arbitration and Control

- **arbiter_monbus_common** - Common arbitration logic for monitoring bus
- **arbiter_rr_pwm_monbus** - Round-robin PWM arbiter for monbus
- **arbiter_wrr_pwm_monbus** - Weighted round-robin PWM arbiter
- **monbus_arbiter** - Monitoring bus arbiter

16.4.3 AXI Monitoring and Analysis

- **axi_monitor_base** - Base AXI/AXIL transaction monitor (per-DMA)
- **axi_monitor_reporter** - Thin dispatcher + 6 packet-type sub-blocks (error / timeout / compl / threshold / perf / debug) gated by ENABLE_*_LOGIC
- **axi_monitor_timeout** - AXI timeout detection
- **axi_monitor_timer** - AXI timing analysis
- **axi_monitor_trans_mgr** - AXI transaction-table management (CAM-backed)
- **monitor_trans_cam** - Multi-port ID CAM with opaque payload (backs trans_mgr)
- **axi_monitor_addr_check** - Per-channel address-match watchpoints
- **axi_monitor_lite** - The AXI/AXIL transaction monitor at a fifth of the gates; behind every front-side `_monlite` wrapper
- **axis_monitor_lite** - The AXI4-Stream monitor in the lite discipline: packets, stalls, bubbles, TID/TDEST changes as Stream/Credit/Channel/Error/Timeout/Completion packets
- **axi4ace_snoop_monitor_lite** - The ACE snoop-channel lite monitor: tracks snoops in AC-issue order because snoop channels have no ID; behind the two ACE snoop-side `_monlite` wrappers

- **axi4_intf_master_observer** - Standalone, protocol-agnostic interface observer (wraps any AXI4 master interface from outside; companion to the per-DMA `axi_monitor_*` family). Includes `axi_bus_meter` and `axi_perf_latency_hist` per port. Lives at `projects/components/utility-ip/misc/rtl/axi4_intf_master_observer.sv`; the `axi4_dma_observer` copy in `rtl/amba/shared/` was retired 2026-08-14.

16.4.4 Monitor Bus Delivery + Bulk-Trace Compression

Path from in-flight monitor packets to host-visible memory + CPU IRQs.

- **monbus_group (family)** - Protocol-agnostic core + four protocol-permutation wrappers:
  - `monbus_axil4_axil4_group` (AXIL slave-read + AXIL master-write, replaces legacy `monbus_axil_group`)
  - `monbus_axil4_axi4_group` (AXIL slave-read + AXI4 burst master-write)
  - `monbus_axi4_axil4_group` (AXI4 burst slave-read + AXIL master-write)
  - `monbus_axi4_axi4_group` (burst on both sides)
- **monbus_group_core_1wr** - The core with ONE master-write port and no slave-read side: error-selected packets go to their own address window as raw records (never compressed), everything else to the trace window; `irq_out` pulses once per whole error record landed and `err_rec_count` counts them. Wrappers: monbus_axil4_group_1wr (AXIL single-beat master-write) and monbus_axi4_group_1wr (AXI4 burst master-write). Built from monbus_burst_writer (the core's writer, lifted out) x2, monbus_wr_mux2 (AW-ordered 2:1 write merge, errors first) and monbus_record_expander
- **monbus_compressor** - Bulk-trace encoder, 2-stage pipeline, per-template `delta_ts`, bit-exact to Python golden (~2.6× ratio)
- **monbus_cam** - 32-entry LRU CAM with per-entry 24b `last_ts` (backs the compressor)
- **monbus_pkt_tally** - on-chip packet-type coverage histogram: SRAM count matrix keyed by `{protocol, pkt_type, event_code}`, 32-entry LRU write-combining cache in front (reuses `monbus_cam`), saturating counts, freeze/flush snapshot + first-event latch

16.4.5 Memory / BRAM Slave (sdpram_slave family)

- **sdpram_slave (family)** - Protocol-agnostic BRAM backend + four protocol-permutation wrappers (`sdpram_slave_axi4_axi4`, `sdpram_slave_axi4_axil`, `sdpram_slave_axil_axi4`, `sdpram_slave_axil_axil`). Canonical memory-dump-ring backend for the monbus group's master-write port.

16.4.6 Address and Data Processing

- **axi_gen_addr** - AXI address generation utility
- **axi_split_combi** - AXI combinational splitting logic
- **axi_master_rd_splitter** - AXI read master splitter
- **axi_master_wr_splitter** - AXI write master splitter

16.4.7 Clock and Reset Management

- **amba_clock_gate_ctrl** - AMBA-specific clock gating controller
- **Clock Domain Crossing** - Single reference for all CDC: decision guide, reset behavior, and every building block (`cdc_synchronizer`, `cdc_open_loop`, `cdc_2_phase_handshake`, `cdc_4_phase_handshake`, async FIFO pointers)

16.5 Protocol Bridge and Shims

16.5.1 AXI4 to APB Conversion

- **axi4_to_apb4_convert** - AXI4 to APB protocol converter
- **axi4_to_apb4_shim** - Lightweight AXI4 to APB shim

16.6 Ocelus

Start with **The Ocelus** – the architecture and what it can do: error/debug/perf packet production for protocols *and* for custom blocks like arbiters, source-side filtering, the two transports, and the three capture strategies (bulk trace, compressed trace, on-chip counting).

16.7 Package Definitions

16.7.1 Documented packages

- **ocelus_amba4_pkg** - AMBA4 monitor types and packet layout
- **ocelus_amba5_pkg** - AMBA5 monitor types and packet layout
- **ocelus_arbiter_pkg** - Monitor-bus arbitration types
- **monitor_package_spec** - The package family, specified

16.7.2 The other two packages are documented, just not under their own names

`ocelus_common_pkg` and `ocelus_pkg` have no same-named page and do not need one – monitor_package_spec IS their page:

- **`ocelus_common_pkg`** (165 importers, the workhorse) is the subject of that whole document – the 128-bit record, the protocol and packet-type enums, the type aliases, the helper functions, the side-band timestamp.
- **`ocelus_pkg`** (30 importers) defines nothing of its own. It is a backward-compatibility shim that re-exports the four split packages, and it is covered by that page's *Backward Compatibility* section.

A second page per package would be a second copy of the same content, which is how documentation rots. Measured 2026-08-31; do not re-raise this as a gap on the strength of a filename search.

`apb4_pkg.sv`, `apb5_pkg.sv`, `axi_pkg.sv` and `bus_types.svh` were DELETED the same day – all four had zero importers (`apb4_pkg` was imported only by `apb5_pkg`, and `bus_types.svh`, which `included` two of them, was included by nothing). They were a vestigial type library, not undocumented code. (`monitor_network_pkg` never had RTL at all – it was always a phantom.)

16.8 Quick Reference

16.8.1 Module Count by Protocol

Protocol	AMBA 4	AMBA 5	Total
APB	9 modules	9 modules	18
AXI (Full)	14 modules	16 modules	30
AXI-Lite	8 modules	-	8
AXI-Stream	4 modules	4 modules	8
GAXI Infrastructure	5 modules	-	5
Shared Utilities	15 modules	-	15
Protocol Bridges	2 modules	-	2

Total: 86 AMBA modules

16.8.2 Performance Characteristics

Protocol	Typical Frequency	Throughput	Use Case
APB4/APB5	100-200 MHz	400 MB/s - 1.6 GB/s	Control registers, low-speed peripherals
AXI4-Lite	200-400 MHz	800 MB/s - 3.2 GB/s	Register access, configuration
AXI4/AXI5	300-500 MHz	9.6 GB/s - 32 GB/s	High-performance memory access
AXI4/5-Stream	400-600 MHz	12.8 GB/s - 76.8 GB/s	Data streaming, packet processing

16.8.3 AMBA 4 vs AMBA 5 Selection Guide

Requirement	Recommendation
Proven, mature design	AMBA 4
Atomic operations needed	AMBA 5 (AXI5)
Memory tagging (MTE)	AMBA 5 (AXI5)
Power management wake-up	AMBA 5 (APB5, AXI5-Stream)
TrustZone integration	AMBA 5 (APB5 PNSE)
Maximum compatibility	AMBA 4
Next-generation features	AMBA 5

16.9 Usage Guidelines

16.9.1 Protocol Selection

- **APB4/APB5**: Simple register access, low area/power requirements
- **AXI4-Lite**: Standard register interface, moderate performance
- **AXI4/AXI5**: High-performance memory access, burst transactions
- **AXI4/5-Stream**: High-throughput data streaming, packet processing

16.9.2 Clock Gating Benefits

- **Power Reduction**: 20-40% dynamic power savings
- **Conditional Operation**: Clock gating when interfaces are idle
- **Fine-Grained Control**: Per-channel clock gating available

16.9.3 Design Integration

```
// Example: Complete AXI5 system with monitoring
axi5_master_rd_mon #(.AXI_DATA_WIDTH(64)) u_rd_master (...);
axi5_master_wr_mon #(.AXI_DATA_WIDTH(64)) u_wr_master (...);
axi5_slave_rd      #(.AXI_DATA_WIDTH(64)) u_rd_slave  (...);
axi5_slave_wr      #(.AXI_DATA_WIDTH(64)) u_wr_slave  (...);
```

16.10 References

- Protocol specifications (Arm IHI 0022 Issues H and L, IHI 0024, IHI 0051, Wishbone B4), the papers behind the shared infrastructure (latency-insensitive and elastic channels, CRC-32, async FIFOs, clock gating), textbook chapters, and related work for the monitor subsystem (an original design) are indexed in `rtl/amba/references/README.md`, with open-access PDFs archived beside it. Per-module pages carry the citations that apply to them.

16.11 Navigation

- **Back to RTL Documentation** - Return to main RTL documentation index